Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Военный факультет

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО

Вариант 4

Выполнил курсант гр. 434701 А. И. Дамарацкий

Проверил О. М. Внук

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы:

– изучить методы работы с интегрированной средой разработки Microsoft Visual Studio Code при использовании объектно-ориентированного языка программирования C++;

– научиться разрабатывать в выбранной среде приложение с графическим интерфейсом;

– изучить MVC с активной и пассивной моделью.

Задачи работы:

– выбрать объектно-ориентированный язык программирования и среду разработки;

– установить и настроить программное обеспечение, ознакомиться с его функциональными возможностями и средствами отладки, привести простые примеры работы с функционалом и продемонстрировать работу отладчика;

– создать пустой проект под выданный вариант задания, установить расширения для работы с SysML v2 и UML;

– разработать каркас MVC с выданной по варианту моделью.

Задание по варианту 4: разработать калькулятор веса на разных планетах. Человек вводит свой вес на Земле, программа должна рассчитать, сколько он будет весить на других планетах Солнечной системы (Меркурий, Нептун и т. д.). На главной форме должна быть кнопка «Ввести данные», по нажатию на которую открывается дополнительное окно для ввода информации. Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отображается результат вычислений. Если введены некорректные данные – показать сообщение об ошибке. Добавить каркас MVC с активной моделью.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Интегрированная среда разработки (IDE, Integrated Development Environment) – комплекс программных средств, объединяющий в одном приложении инструменты, необходимые на всех этапах создания программы. В состав IDE обычно входят:

– редактор исходного кода с подсветкой синтаксиса, автодополнением и навигацией по коду;

– компилятор или интерпретатор языка программирования;

– средства автоматизации сборки;

– отладчик;

– средства управления проектом и интеграции с системой управления версиями;

– система расширений, позволяющая добавлять поддержку новых языков и инструментов.

Microsoft Visual Studio Code – бесплатный кроссплатформенный редактор кода с функциями интегрированной среды разработки. Поддержка конкретного языка подключается расширениями из магазина Visual Studio Marketplace. Для языка C++ используется расширение C/C++ компании Microsoft, которое добавляет анализ кода IntelliSense и интеграцию с отладчиками GDB, LLDB и Microsoft C++. Сборка выполняется задачами (файл tasks.json), запуск под отладчиком – по конфигурациям запуска (файл launch.json) [1].

Отладчик позволяет выполнять программу под контролем: останавливать ее в точках останова, выполнять пошагово, просматривать значения переменных и стек вызовов, вычислять выражения в контексте остановки. Отладчик GDB (GNU Debugger) входит в состав набора компиляторов GCC и используется средой через протокол Machine Interface.

UML (Unified Modeling Language) – унифицированный язык графического описания для объектного моделирования программных систем; диаграмма классов показывает классы системы, их атрибуты, методы и связи между ними. SysML v2 (Systems Modeling Language) – язык моделирования систем, вторая версия которого имеет текстовую нотацию: части системы (part def), атрибуты (attribute def), порты (port def), действия (action def) и требования (requirement def) описываются в текстовом файле [2].

MVC (Model – View – Controller) – архитектурный шаблон, разделяющий приложение на три компонента:

– модель (Model) – данные и бизнес-логика, не зависящие от способа отображения;

– представление (View) – отображение данных модели пользователю;

– контроллер (Controller) – обработка действий пользователя и изменение модели.

Различают два варианта шаблона. Пассивная модель не знает о представлениях: после ее изменения контроллер сам передает новые данные представлению. Активная модель самостоятельно оповещает подписанные на нее представления об изменении своего состояния, как правило с помощью шаблона «Наблюдатель» (Observer): представление регистрируется в модели как слушатель, а модель при каждом изменении вызывает у всех слушателей метод обновления [3].

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Выбор языка программирования и среды разработки

Для выполнения работы выбран объектно-ориентированный язык C++ (стандарт C++17) [4] и среда разработки Microsoft Visual Studio Code 1.141.2. Компиляция выполняется компилятором GCC 16.2.0 из сборки MinGW-w64 (WinLibs), отладка – отладчиком GDB 18.1. Графический интерфейс реализован средствами Windows API [5], которые входят в состав операционной системы и не требуют подключения сторонних библиотек. Состав использованного программного обеспечения приведен в таблице 3.1.

Таблица 3.1 – Состав программного обеспечения

Программное средство	Версия	Назначение
Microsoft Visual Studio Code	1.141.2	Редактирование, сборка, отладка
Расширение C/C++	1.34.4	Анализ кода C++, интеграция с GDB
Компилятор GCC (MinGW-w64)	16.2.0	Компиляция программы на C++
Отладчик GDB	18.1	Отладка программы
Расширение PlantUML	2.18.1	Построение диаграмм UML
Расширение Syside Editor	0.11.0	Поддержка языка SysML v2

Из таблицы 3.1 видно, что все средства разработки, кроме самой среды, подключаются к Visual Studio Code расширениями или внешними программами: компилятор и отладчик – через расширение C/C++, языки моделирования – через расширения PlantUML и Syside Editor.

3.2 Установка и настройка программного обеспечения

Среда Visual Studio Code и компилятор GCC установлены с официальных источников. Каталог bin компилятора добавлен в переменную среды PATH, что проверено командами g++ --version и gdb --version во встроенном терминале среды. В среде установлено расширение C/C++ (ms-vscode.cpptools).

Для сборки в каталоге .vscode проекта создан файл tasks.json с задачей «Сборка (g++)», которая компилирует исходные файлы с ключами -std=c++17 (стандарт языка), -g (отладочная информация), -municode и -mwindows (оконное приложение Windows с точкой входа wWinMain). Для отладки создан файл launch.json с конфигурацией «Отладка (gdb)», которая

перед запуском выполняет сборку. Главное окно среды с открытым проектом приведено на рисунке 3.1.

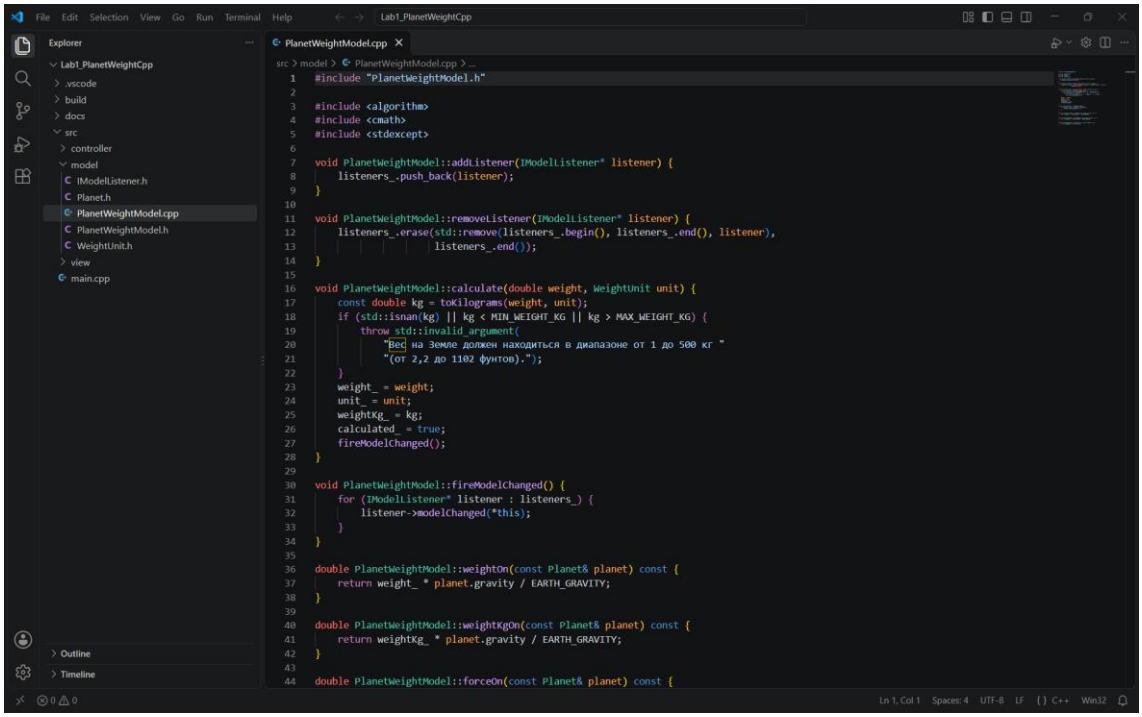


Рисунок 3.1 – Главное окно Visual Studio Code с открытым проектом

Примеры работы с функционалом среды:

– сборка проекта сочетанием клавиш Ctrl+Shift+B, результат которой выводится во встроенный терминал (рисунок 3.2);

– переход к определению функции (F12) и поиск всех ссылок (Shift+F12);

– автодополнение IntelliSense (Ctrl+Space) и подсказка параметров функций Windows API;

– быстрое открытие файла по имени (Ctrl+P);

– переименование идентификатора во всех файлах проекта (F2).



Рисунок 3.2 – Результат сборки проекта

Отладчик запускается клавишей F5. Для демонстрации его работы точка останова установлена в методе WeightController::onInputData() на строке передачи данных модели. После ввода веса 75 кг выполнение программы остановилось на этой строке (рисунок 3.3). На панели Variables отображается значение локальной переменной weight = 75, на панели Call Stack – цепочка вызовов от оконной процедуры MainWindow::windowProc() до обработчика кнопки.

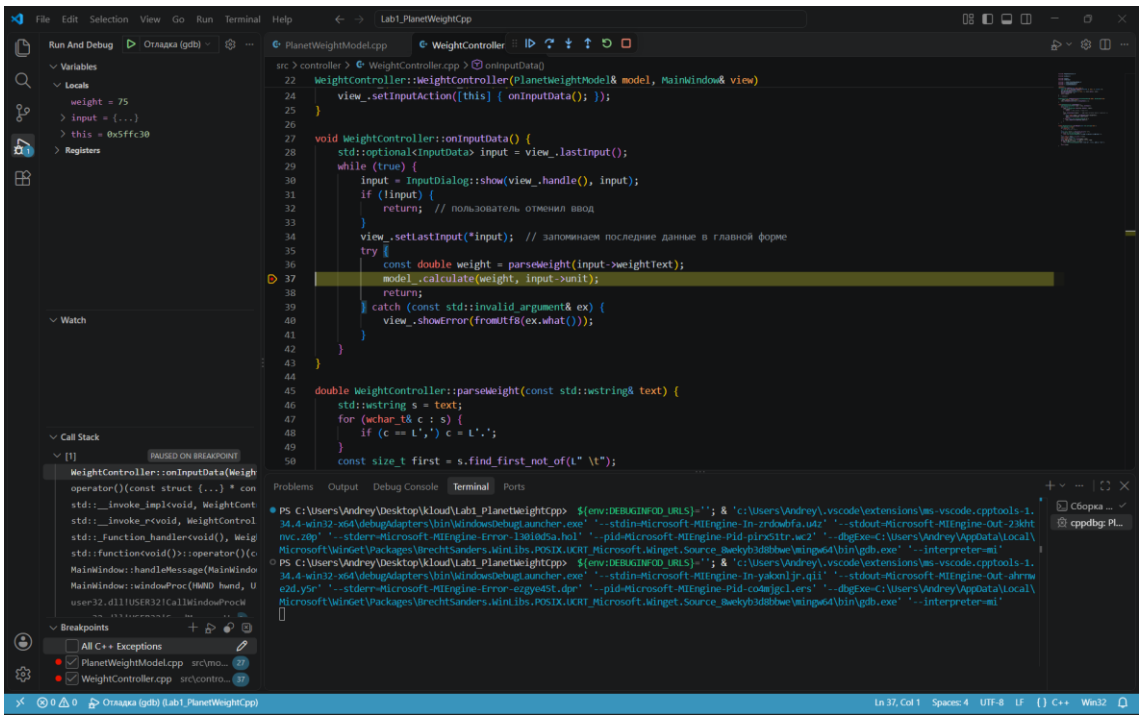


Рисунок 3.3 – Остановка в методе WeightController::onInputData()

Вторая точка останова установлена в методе PlanetWeightModel::calculate() на строке оповещения слушателей. После продолжения выполнения (F5) программа остановилась на ней, при этом на панели Variables раскрыт объект this с полями модели, на панели Watch добавлено выражение this->weightKg_, а в окне Debug Console вычислено выражение this->weightKgOn(solarSystem()[4]) – вес на Юпитере, равный 189,526 кг (рисунок 3.4). Пошаговое выполнение выполняется клавишами F10 (шаг с обходом) и F11 (шаг с заходом).

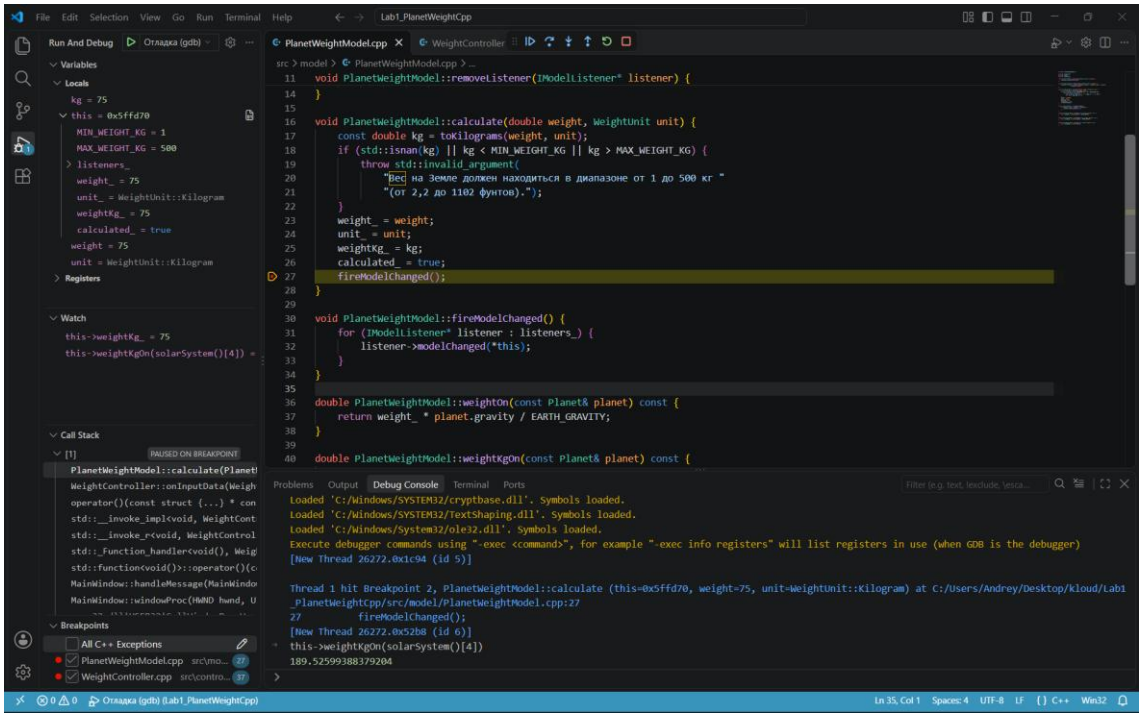


Рисунок 3.4 – Остановка в методе PlanetWeightModel::calculate()

3.3 Создание проекта и установка расширений для SysML v2 и UML

Проект создан как каталог Lab1_PlanetWeightCpp, открытый в среде командой File → Open Folder. Исходные тексты размещены в каталоге src и разделены по компонентам шаблона MVC: model, view и controller. Диаграммы и модель системы размещены в каталоге docs.

Через панель Extensions (Ctrl+Shift+X) установлены расширения, предложенные в задании для Visual Studio Code: PlantUML (jebbs.plantuml) для работы с UML и Syside Editor: SysML v2 Essential (sensmetry.syside-editor) для работы с SysML v2. Расширение PlantUML добавляет подсветку синтаксиса файлов .puml и окно предварительного просмотра диаграмм (Alt+D), расширение Syside Editor – подсветку синтаксиса и проверку файлов .sysml. Список установленных расширений приведен на рисунке 3.5.

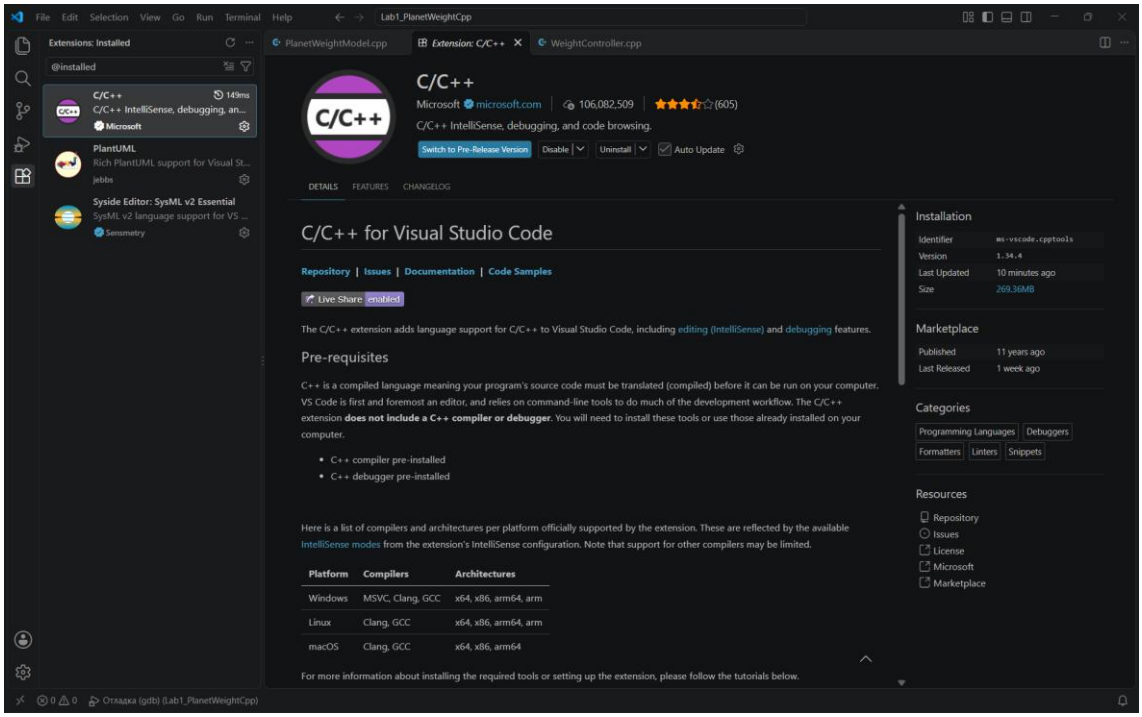


Рисунок 3.5 – Установленные расширения Visual Studio Code

Диаграмма классов приложения описана в нотации PlantUML в файле docs/class-diagram.puml и отображается расширением в окне предварительного просмотра (рисунок 3.6).

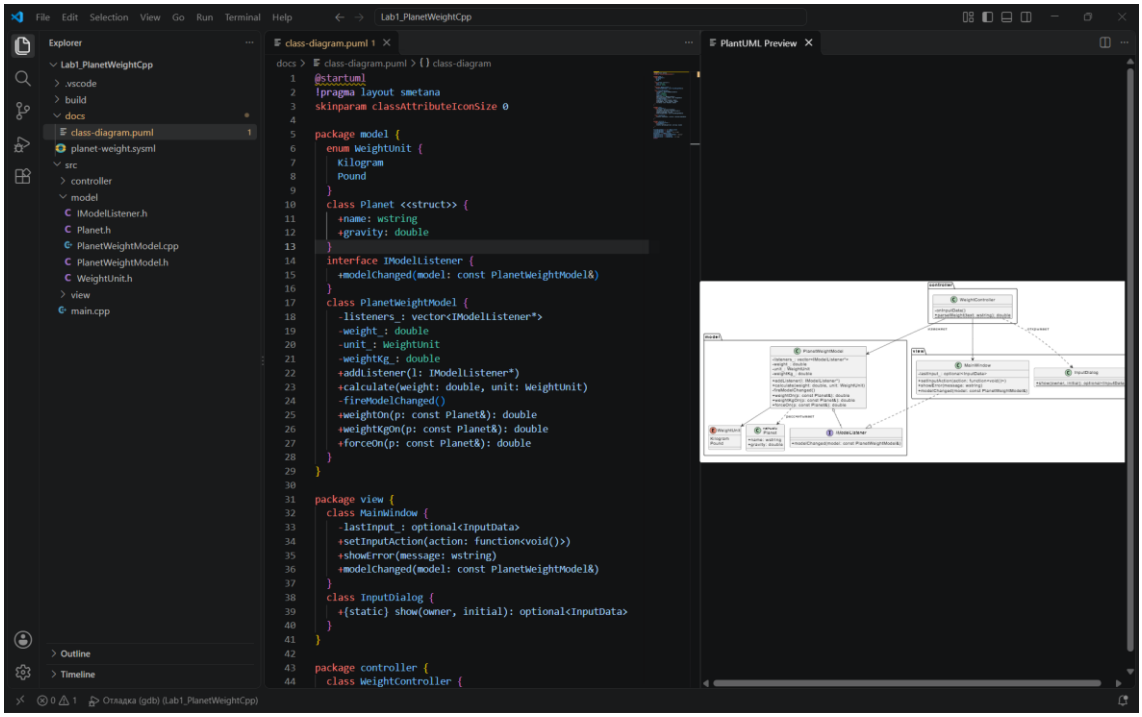


Рисунок 3.6 – Диаграмма классов в окне предварительного просмотра PlantUML

Модель системы в нотации SysML v2 описана в файле docs/planet-weight.sysml: определены единицы измерения веса и планеты, части системы weightModel, mainWindow, inputDialog и weightController, порт уведомления модели и действие расчета веса. Файл в редакторе с подсветкой синтаксиса расширения приведен на рисунке 3.7.

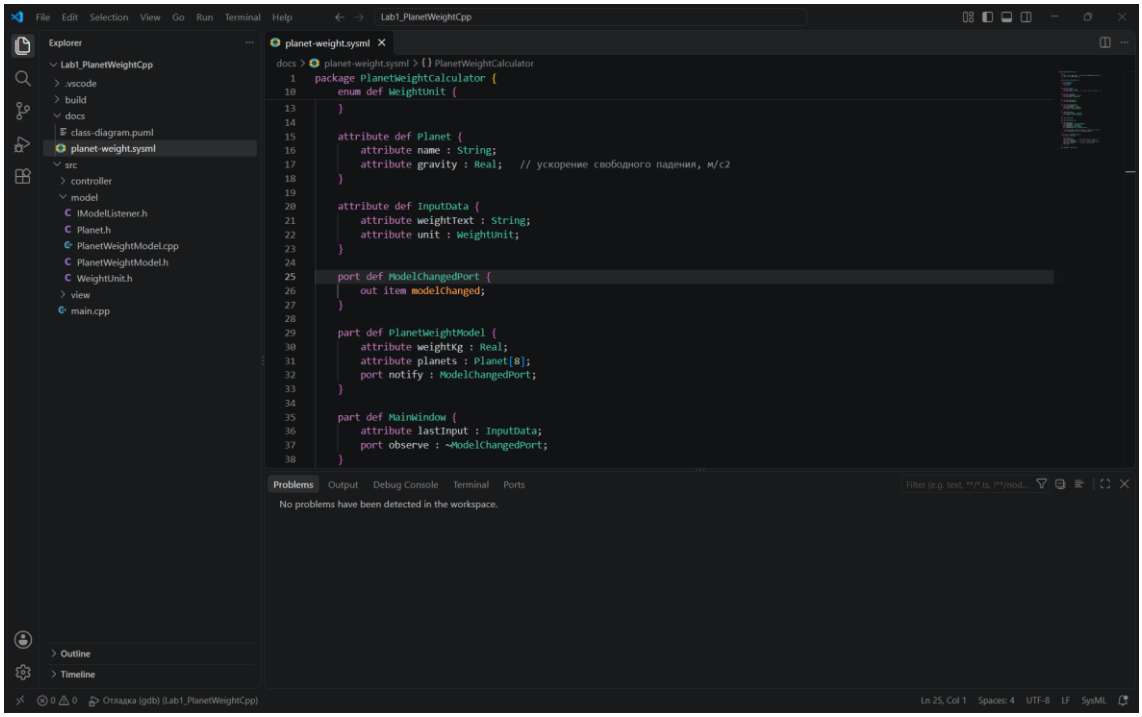


Рисунок 3.7 – Модель системы в нотации SysML v2

3.4 Каркас MVC с активной моделью

Приложение построено по шаблону MVC с активной моделью. Классы приложения и их связи приведены на диаграмме классов (см. рисунок 3.6). Назначение классов приведено в таблице 3.2.

Таблица 3.2 – Классы приложения

Класс	Компонент MVC	Назначение
PlanetWeightModel	Модель	Хранение и проверка веса, расчет, оповещение слушателей
IModelListener	Модель	Интерфейс наблюдателя модели
Planet, WeightUnit	Модель	Планеты Солнечной системы и единицы веса
MainWindow	Представление	Главная форма, отображение результатов
InputDialog	Представление	Модальное окно ввода данных
WeightController	Контроллер	Обработка кнопки «Ввести данные»

Из таблицы 3.2 видно, что классы модели не используют функции Windows API: работа с окнами сосредоточена в классах представления, а контроллер связывает их с моделью.

Модель PlanetWeightModel не зависит от классов графического интерфейса. Метод calculate() проверяет, что вес на Земле находится в диапазоне от 1 до 500 кг, сохраняет его и вызывает метод fireModelChanged(), который оповещает всех слушателей через интерфейс IModelListener. Вес на планете P, кг, вычисляется по формуле

$$P = M \cdot g / g_0, \tag{3.1}$$

где M – вес человека на Земле, кг;

g – ускорение свободного падения на поверхности планеты, м/с²;

g_0 – ускорение свободного падения на Земле, равное 9,81 м/с².

Представление MainWindow реализует интерфейс IModelListener и подписывается на модель в функции wWinMain() вызовом model.addListener(&view). В методе modelChanged() главная форма сама считывает результаты из модели. Последние введенные данные хранятся в главной форме в поле lastInput_ и передаются в окно ввода InputDialog при повторном открытии.

Контроллер WeightController назначает обработчик кнопки «Ввести данные». Обработчик открывает окно ввода, сохраняет введенные данные в главной форме, преобразует текст веса в число и передает его модели. При ошибке преобразования или недопустимом значении контроллер выводит сообщение об ошибке и снова открывает окно ввода. Обновлением главной формы контроллер не занимается – это выполняет активная модель.

При запуске программы отображается главная форма с кнопкой «Ввести данные» (рисунок 3.8). По нажатию кнопки открывается дополнительное окно для ввода веса на Земле и единиц измерения (рисунок 3.9).

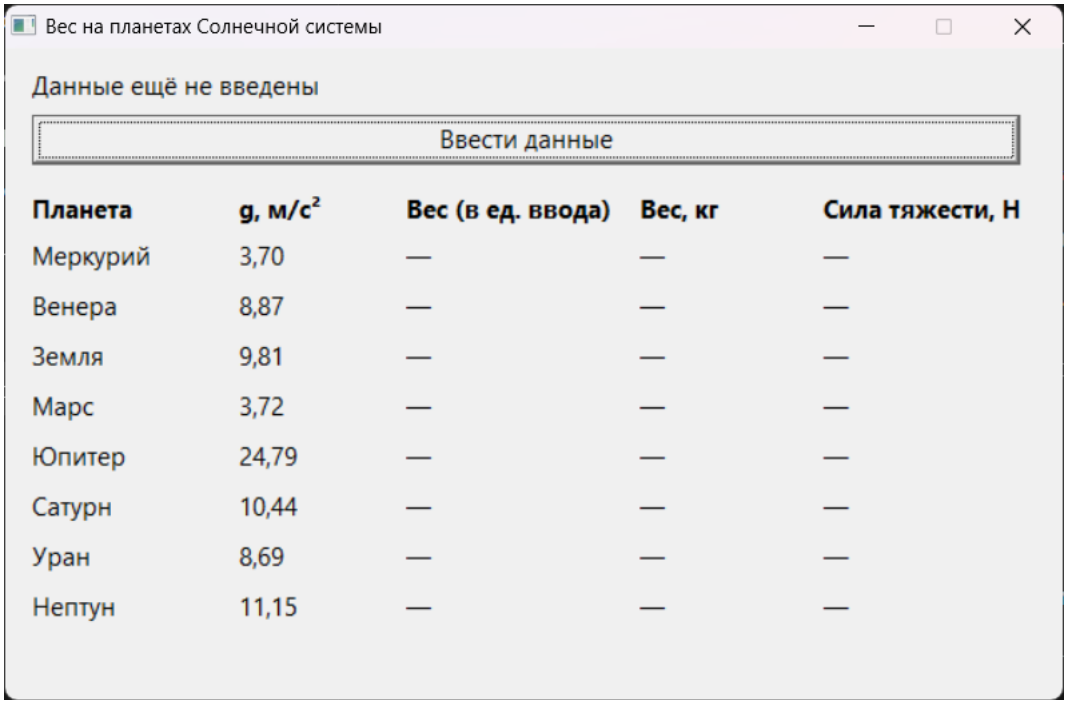


Рисунок 3.8 – Главная форма при запуске программы

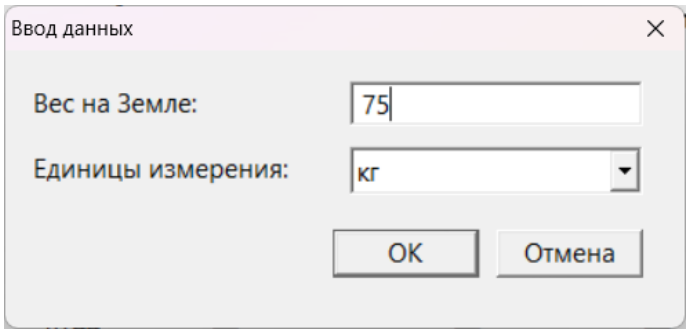


Рисунок 3.9 – Окно ввода данных

После ввода веса 75 кг и нажатия кнопки «OK» модель выполняет расчет и оповещает главную форму, которая отображает вес на восьми планетах Солнечной системы (рисунок 3.10). Например, на Меркурии вес составляет 28,29 кг, на Юпитере – 189,53 кг, на Нептуне – 85,24 кг.

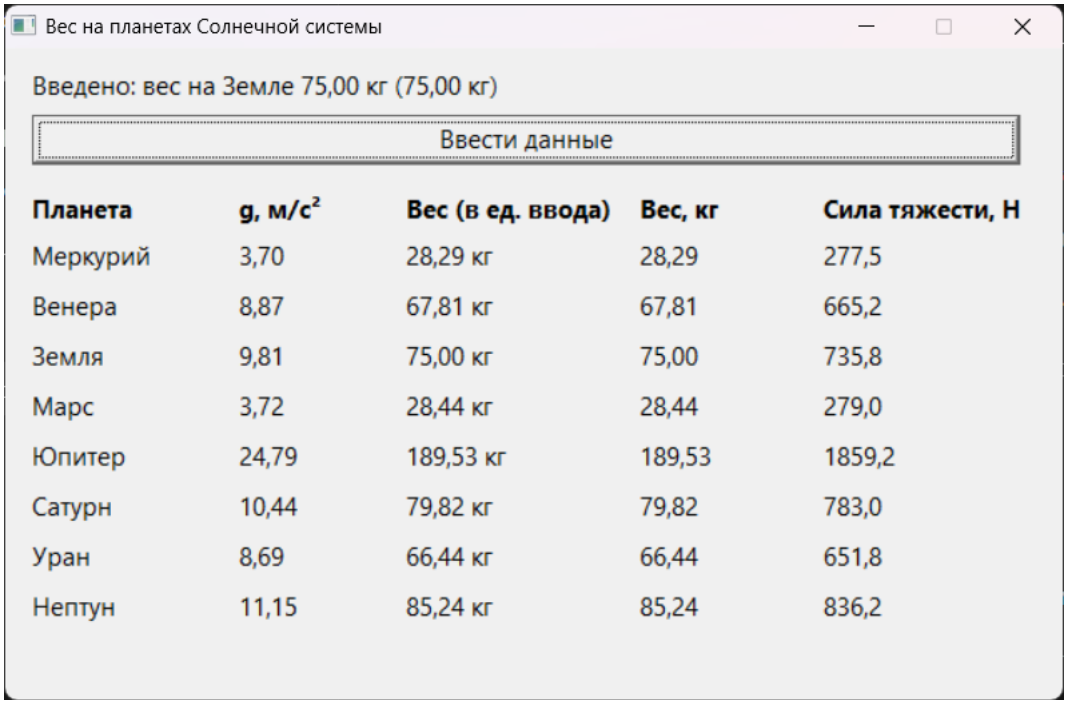


Рисунок 3.10 – Главная форма с результатом вычислений

При повторном нажатии кнопки в окне ввода восстанавливаются последние введенные данные (рисунок 3.11). При вводе текста, не являющегося числом, выводится сообщение об ошибке формата (рисунок 3.12), а при вводе веса вне допустимого диапазона – сообщение, сформированное моделью (рисунок 3.13).

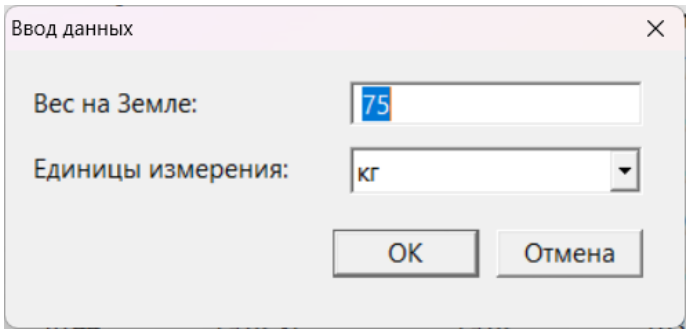


Рисунок 3.11 – Восстановление последних введенных данных

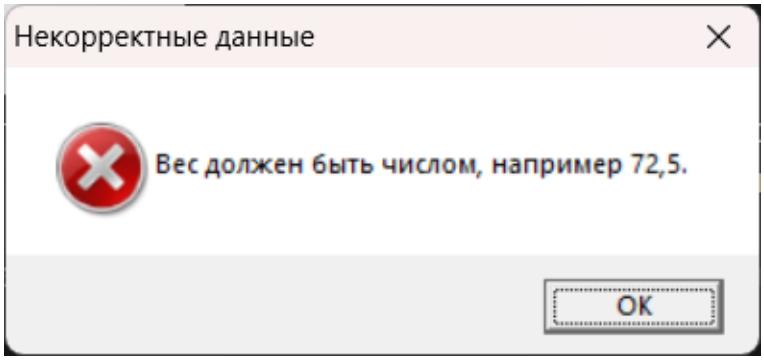


Рисунок 3.12 – Сообщение об ошибке при нечисловом вводе

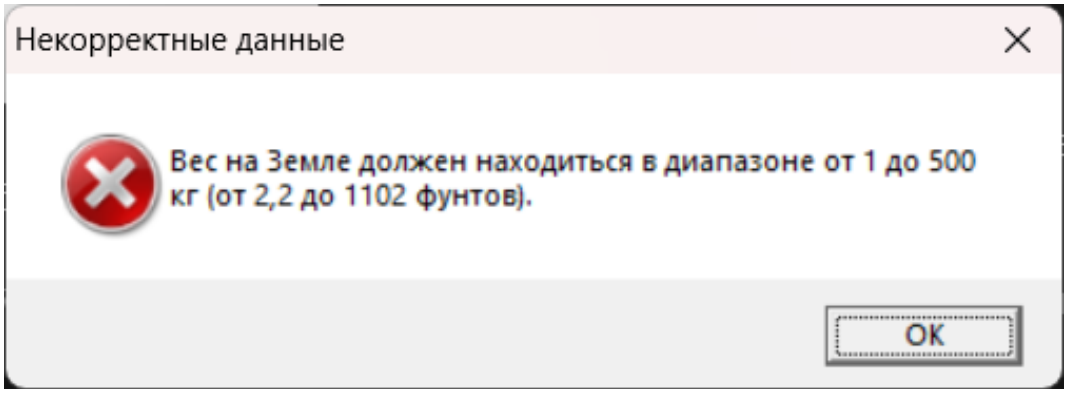


Рисунок 3.13 – Сообщение об ошибке при выходе значения за диапазон

Исходные тексты программы приведены в приложении А.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор исходного кода с подсветкой синтаксиса и автодополнением, компилятор или интерпретатор языка, средства автоматизации сборки, отладчик, средства управления проектом, интеграция с системой управления версиями, встроенный терминал и система расширений. В Visual Studio Code компилятор и отладчик подключаются внешними инструментами (GCC, GDB) через расширение C/C++.

2 Каковы основные приемы работы с редактором кода?

Основные приемы: автодополнение (Ctrl+Space), переход к определению (F12) и поиск ссылок (Shift+F12), быстрое открытие файла (Ctrl+P), палитра команд (Ctrl+Shift+P), переименование идентификатора (F2), форматирование документа (Shift+Alt+F), комментирование строк (Ctrl+/), множественные курсоры (Alt+щелчок), сворачивание блоков кода.

3 Каким образом выполняется сборка и запуск проекта?

Сборка выполняется задачей из файла .vscode/tasks.json (Ctrl+Shift+B): среда вызывает компилятор g++, который компилирует исходные файлы и компонует исполняемый файл build/PlanetWeight.exe. Запуск под отладчиком выполняется клавишей F5 по конфигурации из файла .vscode/launch.json, запуск без отладки – сочетанием Ctrl+F5; перед запуском автоматически выполняется сборка.

4 Что такое MVC?

MVC (Model – View – Controller) – архитектурный шаблон, разделяющий приложение на модель (данные и бизнес-логика), представление (отображение данных) и контроллер (обработка действий пользователя). Разделение уменьшает связанность компонентов, позволяет изменять интерфейс без изменения логики и иметь несколько представлений одной модели.

5 Чем отличается активная модель от пассивной?

Пассивная модель не знает о представлениях: после ее изменения контроллер сам обновляет представление. Активная модель самостоятельно оповещает подписанные на нее представления об изменении состояния по шаблону «Наблюдатель», а контроллер только передает ей входные данные.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены методы работы с интегрированной средой разработки Microsoft Visual Studio Code при использовании языка C++: установка и настройка компилятора GCC и отладчика GDB, настройка задач сборки и конфигураций запуска, основные приемы работы с редактором кода. Изучены средства отладки: установка точек останова, просмотр значений переменных и стека вызовов, вычисление выражений, пошаговое выполнение.

Установлены расширения для работы с UML и SysML v2, построены диаграмма классов приложения и модель системы в текстовой нотации SysML v2.

Изучен шаблон MVC с активной и пассивной моделью. Разработано приложение с графическим интерфейсом – калькулятор веса на разных планетах – на каркасе MVC с активной моделью: модель самостоятельно оповещает главную форму об изменении данных, а контроллер только передает модели введенные пользователем данные.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Visual Studio Code. Documentation [Электронный ресурс]. – Режим доступа : https://code.visualstudio.com/docs.

2 OMG Systems Modeling Language (SysML) Version 2 [Электронный ресурс]. – Режим доступа : https://www.omg.org/spec/SysML/2.0/.

3 Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – СПб. : Питер, 2020. – 448 с.

4 Страуструп, Б. Язык программирования C++ / Б. Страуструп. – 4-е изд. – М. : Бином, 2015. – 1136 с.

5 Win32 API [Электронный ресурс] / Microsoft. – Режим доступа : https://learn.microsoft.com/windows/win32/.

6 Жизненный цикл разработки программного обеспечения : лабораторная работа № 1 «Интегрированная среда разработки ПО». – Минск : БГУИР, 2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходные тексты программы

Файл main.cpp

```
#include <windows.h>

#include "controller/WeightController.h"
#include "model/PlanetWeightModel.h"
#include "view/MainWindow.h"

// Точка входа: создание и связывание компонентов MVC.
int WINAPI wWinMain(HINSTANCE instance, HINSTANCE, PWSTR, int showCommand) {
    PlanetWeightModel model;
    MainWindow view;
    if (!view.create(instance, showCommand)) {
        return 1;
    }
    model.addListener(&view);  // представление подписывается на активную
модель
    WeightController controller(model, view);

    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        if (!IsDialogMessageW(view.handle(), &msg)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }
    return static_cast<int>(msg.wParam);
}
```

Файл model/WeightUnit.h

```
#pragma once

#include <string>

// Единицы измерения веса, поддерживаемые приложением.
enum class WeightUnit {
    Kilogram,
    Pound
};

// Количество килограммов в одной единице измерения.
inline double kilogramsPerUnit(WeightUnit unit) {
    return unit == WeightUnit::Kilogram ? 1.0 : 0.45359237;
}

inline double toKilograms(double value, WeightUnit unit) {
    return value * kilogramsPerUnit(unit);
}

inline double fromKilograms(double kilograms, WeightUnit unit) {
    return kilograms / kilogramsPerUnit(unit);
}
```

```
inline std::wstring unitTitle(WeightUnit unit) {
    return unit == WeightUnit::Kilogram ? L"кг" : L"фунты";
}
```

Файл model/Planet.h

```
#pragma once

#include <array>
#include <string>

// Планета Солнечной системы и ускорение свободного падения
// на ее поверхности, м/с2.
struct Planet {
    std::wstring name;
    double gravity;
};

constexpr double EARTH_GRAVITY = 9.81;

// Планеты Солнечной системы от Меркурия до Нептуна.
inline const std::array<Planet, 8>& solarSystem() {
    static const std::array<Planet, 8> planets = {{
        {L"Меркурий", 3.70},
        {L"Венера", 8.87},
        {L"Земля", 9.81},
        {L"Марс", 3.72},
        {L"Юпитер", 24.79},
        {L"Сатурн", 10.44},
        {L"Уран", 8.69},
        {L"Нептун", 11.15},
    }};
    return planets;
}
```

Файл model/IModelListener.h

```
#pragma once

class PlanetWeightModel;

// Интерфейс наблюдателя активной модели. Представление реализует его,
// чтобы получать уведомления об изменении состояния модели.
class IModelListener {
public:
    virtual ~IModelListener() = default;
    virtual void modelChanged(const PlanetWeightModel& model) = 0;
};
```

Файл model/PlanetWeightModel.h

```
#pragma once

#include <vector>

#include "IModelListener.h"
#include "Planet.h"
#include "WeightUnit.h"
```

```
// Активная модель: хранит вес человека на Земле, рассчитывает его вес
// на других планетах и сама уведомляет подписанные представления.
class PlanetWeightModel {
public:
    static constexpr double MIN_WEIGHT_KG = 1.0;
    static constexpr double MAX_WEIGHT_KG = 500.0;

    void addListener(IModelListener* listener);
    void removeListener(IModelListener* listener);

    // Проверяет вес на Земле, сохраняет его и оповещает наблюдателей.
    // При недопустимом значении выбрасывает std::invalid_argument.
    void calculate(double weight, WeightUnit unit);

    bool isCalculated() const { return calculated_; }
    double weight() const { return weight_; }
    WeightUnit unit() const { return unit_; }
    double weightKg() const { return weightKg_; }

    // Вес на планете в единицах, в которых он был введен.
    double weightOn(const Planet& planet) const;
    // Вес на планете в килограммах.
    double weightKgOn(const Planet& planet) const;
    // Сила тяжести, действующая на человека на планете, Н.
    double forceOn(const Planet& planet) const;

private:
    void fireModelChanged();

    std::vector<IModelListener*> listeners_;
    double weight_ = 0.0;
    WeightUnit unit_ = WeightUnit::Kilogram;
    double weightKg_ = 0.0;
    bool calculated_ = false;
};
```

Файл model/PlanetWeightModel.cpp

```
#include "PlanetWeightModel.h"

#include <algorithm>
#include <cmath>
#include <stdexcept>

void PlanetWeightModel::addListener(IModelListener* listener) {
    listeners_.push_back(listener);
}

void PlanetWeightModel::removeListener(IModelListener* listener) {
    listeners_.erase(std::remove(listeners_.begin(), listeners_.end(), lis-
tener),
                     listeners_.end());
}

void PlanetWeightModel::calculate(double weight, WeightUnit unit) {
    const double kg = toKilograms(weight, unit);
    if (std::isnan(kg) || kg < MIN_WEIGHT_KG || kg > MAX_WEIGHT_KG) {
        throw std::invalid_argument(
            "Вес на Земле должен находиться в диапазоне от 1 до 500 кг "
```

```
            "(от 2,2 до 1102 фунтов).");
    }
    weight_ = weight;
    unit_ = unit;
    weightKg_ = kg;
    calculated_ = true;
    fireModelChanged();
}

void PlanetWeightModel::fireModelChanged() {
    for (IModelListener* listener : listeners_) {
        listener->modelChanged(*this);
    }
}

double PlanetWeightModel::weightOn(const Planet& planet) const {
    return weight_ * planet.gravity / EARTH_GRAVITY;
}

double PlanetWeightModel::weightKgOn(const Planet& planet) const {
    return weightKg_ * planet.gravity / EARTH_GRAVITY;
}

double PlanetWeightModel::forceOn(const Planet& planet) const {
    return weightKg_ * planet.gravity;
}
```

Файл view/Ui.h

```
#pragma once

#include <windows.h>

#include <string>

// Вспомогательные функции создания элементов управления WinAPI.
namespace ui {

inline HFONT font() {
    static HFONT f = CreateFontW(-15, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE, DEFAULT_CHARSET,
                                 OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
CLEARTYPE_QUALITY,
                                 DEFAULT_PITCH, L"Segoe UI");
    return f;
}

inline HFONT boldFont() {
    static HFONT f = CreateFontW(-15, 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
DEFAULT_CHARSET,
                                 OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
CLEARTYPE_QUALITY,
                                 DEFAULT_PITCH, L"Segoe UI");
    return f;
}

inline HWND create(HWND parent, const wchar_t* cls, const wchar_t* text,
DWORD style, int x, int y,
                   int w, int h, int id = 0, DWORD exStyle = 0) {
```

```
    HWND hwnd = CreateWindowExW(exStyle, cls, text, WS_CHILD | WS_VISIBLE |
style, x, y, w, h,
                                parent, reinter-
pret_cast<HMENU>(static_cast<INT_PTR>(id)),
                                GetModuleHandleW(nullptr), nullptr);
    SendMessageW(hwnd, WM_SETFONT, reinterpret_cast<WPARAM>(font()), TRUE);
    return hwnd;
}

inline HWND label(HWND parent, const wchar_t* text, int x, int y, int w, int
h) {
    return create(parent, L"STATIC", text, SS_LEFT, x, y, w, h);
}

inline HWND edit(HWND parent, int x, int y, int w, int h) {
    return create(parent, L"EDIT", L"", WS_TABSTOP | ES_AUTOHSCROLL, x, y, w,
h, 0,
                  WS_EX_CLIENTEDGE);
}

inline HWND combo(HWND parent, int x, int y, int w, int h) {
    return create(parent, L"COMBOBOX", L"", WS_TABSTOP | CBS_DROPDOWNLIST, x,
y, w, h);
}

inline HWND button(HWND parent, const wchar_t* text, int id, int x, int y,
int w, int h,
                   bool isDefault) {
    return create(parent, L"BUTTON", text, WS_TABSTOP | (isDefault ?
BS_DEFPUSHBUTTON : 0), x, y,
                  w, h, id);
}

inline std::wstring text(HWND hwnd) {
    std::wstring s(GetWindowTextLengthW(hwnd) + 1, L'\0');
    GetWindowTextW(hwnd, s.data(), static_cast<int>(s.size()));
    s.resize(s.size() - 1);
    return s;
}

}  // namespace ui
```

Файл view/InputDialog.h

```
#pragma once

#include <windows.h>

#include <optional>
#include <string>

#include "../model/WeightUnit.h"

// Данные, введенные пользователем. Вес хранится как текст, чтобы
// восстановить в окне даже ошибочный ввод.
struct InputData {
    std::wstring weightText;
    WeightUnit unit = WeightUnit::Kilogram;
};
```

```
// Модальное окно ввода веса на Земле.
class InputDialog {
public:
    // Показывает окно и возвращает введенные данные либо пустое значение,
    // если ввод отменен.
    static std::optional<InputData> show(HWND owner, const
std::optional<InputData>& initial);

private:
    InputDialog(HWND owner, const std::optional<InputData>& initial);
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam);
    LRESULT handleMessage(UINT msg, WPARAM wParam, LPARAM lParam);
    void createControls();
    void accept();

    HWND owner_;
    HWND hwnd_ = nullptr;
    HWND weightEdit_ = nullptr;
    HWND unitCombo_ = nullptr;
    std::optional<InputData> initial_;
    std::optional<InputData> result_;
    bool done_ = false;
};
```

Файл view/InputDialog.cpp

```
#include "InputDialog.h"

#include "Ui.h"

namespace {
constexpr wchar_t CLASS_NAME[] = L"PlanetWeightInputDialog";
constexpr int ID_OK = 1;
constexpr int ID_CANCEL = 2;
}  // namespace

std::optional<InputData> InputDialog::show(HWND owner, const
std::optional<InputData>& initial) {
    static bool registered = false;
    if (!registered) {
        WNDCLASSW wc{};
        wc.lpfnWndProc = windowProc;
        wc.hInstance = GetModuleHandleW(nullptr);
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_BTNFACE + 1);
        wc.lpszClassName = CLASS_NAME;
        RegisterClassW(&wc);
        registered = true;
    }

    InputDialog dialog(owner, initial);
    RECT ownerRect;
    GetWindowRect(owner, &ownerRect);
    const int width = 400, height = 190;
    const int x = ownerRect.left + (ownerRect.right - ownerRect.left - width)
/ 2;
    const int y = ownerRect.top + (ownerRect.bottom - ownerRect.top - height)
/ 2;
    CreateWindowExW(WS_EX_DLGMODALFRAME, CLASS_NAME, L"Ввод данных",
```

```
                    WS_POPUP | WS_CAPTION | WS_SYSMENU | WS_VISIBLE, x, y,
width, height,
                    owner, nullptr, GetModuleHandleW(nullptr), &dialog);

    // Модальность: главное окно недоступно, пока открыто окно ввода.
    EnableWindow(owner, FALSE);
    MSG msg;
    while (!dialog.done_ && GetMessageW(&msg, nullptr, 0, 0)) {
        if (!IsDialogMessageW(dialog.hwnd_, &msg)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }
    EnableWindow(owner, TRUE);
    SetForegroundWindow(owner);
    DestroyWindow(dialog.hwnd_);
    return dialog.result_;
}

InputDialog::InputDialog(HWND owner, const std::optional<InputData>& initial)
    : owner_(owner), initial_(initial) {}

LRESULT CALLBACK InputDialog::windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam) {
    InputDialog* self = nullptr;
    if (msg == WM_NCCREATE) {
        self = stat-
ic_cast<InputDialog*>(reinterpret_cast<CREATESTRUCTW*>(lParam)-
>lpCreateParams);
        self->hwnd_ = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA, reinter-
pret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<InputDialog*>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }
    return self ? self->handleMessage(msg, wParam, lParam) : DefWin-
dowProcW(hwnd, msg, wParam, lParam);
}

LRESULT InputDialog::handleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:
        createControls();
        return 0;
    case WM_COMMAND:
        if (LOWORD(wParam) == ID_OK || LOWORD(wParam) == IDOK) {
            accept();
        } else if (LOWORD(wParam) == ID_CANCEL || LOWORD(wParam) == IDCANCEL)
{
            done_ = true;  // ввод отменен
        }
        return 0;
    case WM_CLOSE:
        done_ = true;
        return 0;
    }
    return DefWindowProcW(hwnd_, msg, wParam, lParam);
}

void InputDialog::createControls() {
    ui::label(hwnd_, L"Вес на Земле:", 16, 20, 170, 24);
```

```
    weightEdit_ = ui::edit(hwnd_, 200, 18, 170, 26);
    ui::label(hwnd_, L"Единицы измерения:", 16, 58, 180, 24);
    unitCombo_ = ui::combo(hwnd_, 200, 56, 170, 120);
    SendMessageW(unitCombo_, CB_ADDSTRING, 0, reinter-
pret_cast<LPARAM>(L"кг"));
    SendMessageW(unitCombo_, CB_ADDSTRING, 0, reinter-
pret_cast<LPARAM>(L"фунты"));
    ui::button(hwnd_, L"OK", ID_OK, 190, 104, 85, 28, true);
    ui::button(hwnd_, L"Отмена", ID_CANCEL, 285, 104, 85, 28, false);

    // Восстановление последних введенных данных.
    if (initial_) {
        SetWindowTextW(weightEdit_, initial_->weightText.c_str());
        SendMessageW(unitCombo_, CB_SETCURSEL, initial_->unit ==
WeightUnit::Kilogram ? 0 : 1, 0);
    } else {
        SendMessageW(unitCombo_, CB_SETCURSEL, 0, 0);
    }
    SendMessageW(weightEdit_, EM_SETSEL, 0, -1);
    SetFocus(weightEdit_);
}

void InputDialog::accept() {
    InputData data;
    data.weightText = ui::text(weightEdit_);
    data.unit = SendMessageW(unitCombo_, CB_GETCURSEL, 0, 0) == 1 ?
WeightUnit::Pound
                                                                  :
WeightUnit::Kilogram;
    result_ = data;
    done_ = true;
}
```

Файл view/MainWindow.h

```
#pragma once

#include <windows.h>

#include <array>
#include <functional>
#include <optional>
#include <string>

#include "../model/IModelListener.h"
#include "InputDialog.h"

// Главная форма. Является наблюдателем активной модели: при изменении
// модели сама считывает из нее данные и обновляет метки.
class MainWindow : public IModelListener {
public:
    bool create(HINSTANCE instance, int showCommand);
    HWND handle() const { return hwnd_; }

    void setInputAction(std::function<void()> action) { inputAction_ =
std::move(action); }
    const std::optional<InputData>& lastInput() const { return lastInput_; }
    void setLastInput(const InputData& input) { lastInput_ = input; }
    void showError(const std::wstring& message) const;
```

```
    void modelChanged(const PlanetWeightModel& model) override;

private:
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam);
    LRESULT handleMessage(UINT msg, WPARAM wParam, LPARAM lParam);
    void createControls();

    HWND hwnd_ = nullptr;
    HWND summaryLabel_ = nullptr;
    HWND inputButton_ = nullptr;
    std::array<HWND, 8> weightLabels_{};
    std::array<HWND, 8> weightKgLabels_{};
    std::array<HWND, 8> forceLabels_{};
    std::function<void()> inputAction_;
    // Последние введенные данные хранятся в главной форме.
    std::optional<InputData> lastInput_;
};
```

Файл view/MainWindow.cpp

```
#include "MainWindow.h"

#include <cstdio>

#include "../model/PlanetWeightModel.h"
#include "Ui.h"

namespace {
constexpr wchar_t CLASS_NAME[] = L"PlanetWeightMainWindow";
constexpr int ID_INPUT = 100;

std::wstring format(const wchar_t* pattern, double value) {
    wchar_t buffer[64];
    swprintf(buffer, 64, pattern, value);
    std::wstring s(buffer);
    for (wchar_t& c : s) {
        if (c == L'.') c = L',';  // десятичная запятая
    }
    return s;
}
}  // namespace

bool MainWindow::create(HINSTANCE instance, int showCommand) {
    WNDCLASSW wc{};
    wc.lpfnWndProc = windowProc;
    wc.hInstance = instance;
    wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_BTNFACE + 1);
    wc.lpszClassName = CLASS_NAME;
    RegisterClassW(&wc);

    hwnd_ = CreateWindowExW(0, CLASS_NAME, L"Вес на планетах Солнечной
системы",
                            WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU |
WS_MINIMIZEBOX,
                            CW_USEDEFAULT, CW_USEDEFAULT, 640, 420, nullptr,
nullptr, instance,
                            this);
    if (!hwnd_) return false;
```

```
    ShowWindow(hwnd_, showCommand);
    UpdateWindow(hwnd_);
    return true;
}

LRESULT CALLBACK MainWindow::windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam) {
    MainWindow* self = nullptr;
    if (msg == WM_NCCREATE) {
        self = stat-
ic_cast<MainWindow*>(reinterpret_cast<CREATESTRUCTW*>(lParam)-
>lpCreateParams);
        self->hwnd_ = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA, reinter-
pret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<MainWindow*>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }
    return self ? self->handleMessage(msg, wParam, lParam) : DefWin-
dowProcW(hwnd, msg, wParam, lParam);
}

LRESULT MainWindow::handleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:
        createControls();
        return 0;
    case WM_SETFOCUS:
        SetFocus(inputButton_);  // кнопка «Ввести данные» нажимается и с
клавиатуры
        return 0;
    case WM_COMMAND:
        if ((LOWORD(wParam) == ID_INPUT || LOWORD(wParam) == IDOK) && in-
putAction_) {
            inputAction_();  // действие пользователя передается контроллеру
        }
        return 0;
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(hwnd_, msg, wParam, lParam);
}

void MainWindow::createControls() {
    summaryLabel_ = ui::label(hwnd_, L"Данные ещё не введены", 16, 12, 600,
24);
    inputButton_ = ui::button(hwnd_, L"Ввести данные", ID_INPUT, 16, 40, 592,
30, true);


    const int columns[] = {16, 140, 240, 380, 490};
    const wchar_t* headers[] = {L"Планета", L"g, м/с²", L"Вес (в ед. ввода)",
L"Вес, кг",
                                L"Сила тяжести, Н"};
    for (int c = 0; c < 5; ++c) {
        HWND h = ui::label(hwnd_, headers[c], columns[c], 86, 130, 24);
        SendMessageW(h, WM_SETFONT, reinterpret_cast<WPARAM>(ui::boldFont()),
TRUE);
    }
    const auto& planets = solarSystem();
    for (size_t i = 0; i < planets.size(); ++i) {
```

```
        const int y = 114 + static_cast<int>(i) * 30;
        ui::label(hwnd_, planets[i].name.c_str(), columns[0], y, 120, 24);
        ui::label(hwnd_, format(L"%.2f", planets[i].gravity).c_str(), col-
umns[1], y, 90, 24);
        weightLabels_[i] = ui::label(hwnd_, L"—", columns[2], y, 130, 24);
        weightKgLabels_[i] = ui::label(hwnd_, L"—", columns[3], y, 100, 24);
        forceLabels_[i] = ui::label(hwnd_, L"—", columns[4], y, 120, 24);
    }
}

void MainWindow::showError(const std::wstring& message) const {
    MessageBoxW(hwnd_, message.c_str(), L"Некорректные данные", MB_OK |
MB_ICONERROR);
}

void MainWindow::modelChanged(const PlanetWeightModel& model) {
    if (!model.isCalculated()) return;
    const std::wstring unit = unitTitle(model.unit());
    SetWindowTextW(summaryLabel_, (L"Введено: вес на Земле " + for-
mat(L"%.2f", model.weight()) +
                                   L" " + unit + L" (" + format(L"%.2f", mod-
el.weightKg()) +
                                   L" кг)").c_str());
    const auto& planets = solarSystem();
    for (size_t i = 0; i < planets.size(); ++i) {
        SetWindowTextW(weightLabels_[i],
                       (format(L"%.2f", model.weightOn(planets[i])) + L" " +
unit).c_str());
        SetWindowTextW(weightKgLabels_[i], format(L"%.2f", mod-
el.weightKgOn(planets[i])).c_str());
        SetWindowTextW(forceLabels_[i], format(L"%.1f", mod-
el.forceOn(planets[i])).c_str());
    }
}
```

Файл controller/WeightController.h

```
#pragma once

#include <string>

class PlanetWeightModel;
class MainWindow;

// Контроллер: обрабатывает действия пользователя, проверяет введенные
// данные и изменяет модель. Представление контроллер не обновляет –
// при активной модели это делает сама модель через механизм уведомлений.
class WeightController {
public:
    WeightController(PlanetWeightModel& model, MainWindow& view);

    // Преобразует текст в число; при ошибке выбрасывает
std::invalid_argument.
    static double parseWeight(const std::wstring& text);

private:
    void onInputData();  // обработчик кнопки «Ввести данные»

    PlanetWeightModel& model_;
    MainWindow& view_;
```

```
};
```

Файл controller/WeightController.cpp

```
#include "WeightController.h"

#include <windows.h>

#include <cwchar>
#include <stdexcept>

#include "../model/PlanetWeightModel.h"
#include "../view/InputDialog.h"
#include "../view/MainWindow.h"

namespace {
std::wstring fromUtf8(const char* text) {
    const int size = MultiByteToWideChar(CP_UTF8, 0, text, -1, nullptr, 0);
    std::wstring result(size, L'\0');
    MultiByteToWideChar(CP_UTF8, 0, text, -1, result.data(), size);
    result.resize(size - 1);
    return result;
}
}  // namespace

WeightController::WeightController(PlanetWeightModel& model, MainWindow&
view)
    : model_(model), view_(view) {
    view_.setInputAction([this] { onInputData(); });
}

void WeightController::onInputData() {
    std::optional<InputData> input = view_.lastInput();
    while (true) {
        input = InputDialog::show(view_.handle(), input);
        if (!input) {
            return;  // пользователь отменил ввод
        }
        view_.setLastInput(*input);  // запоминаем последние данные в главной
форме
        try {
            const double weight = parseWeight(input->weightText);
            model_.calculate(weight, input->unit);
            return;
        } catch (const std::invalid_argument& ex) {
            view_.showError(fromUtf8(ex.what()));
        }
    }
}

double WeightController::parseWeight(const std::wstring& text) {
    std::wstring s = text;
    for (wchar_t& c : s) {
        if (c == L',') c = L'.';
    }
    const size_t first = s.find_first_not_of(L" \t");
    if (first == std::wstring::npos) {
        throw std::invalid_argument("Поле «Вес на Земле» не заполнено.");
    }
    const wchar_t* begin = s.c_str() + first;
    wchar_t* end = nullptr;
```

```
    const double value = std::wcstod(begin, &end);
    while (end && (*end == L' ' || *end == L'\t')) ++end;
    if (end == begin || *end != L'\0') {
        throw std::invalid_argument("Вес должен быть числом, например
72,5.");
    }
    return value;
}
```